

МИНОБРНАУКИ РОССИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕ-
ЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
“ВОРОНЕЖСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ”
(ФГБОУ ВО «ВГУ»)

Факультет компьютерных наук

Кафедра программирования и информационных технологий

Создание мобильного приложения «CS Schedule»

Курсовая работа

09.03.02 Информационные системы и технологии

Профиль «Программная инженерия в информационных системах»

Зав. кафедрой _____ д. ф-м.н., профессор С.Д. Махортов

Обучающийся _____ ст. 3 курса оч. отд. Я.А. Ольховский

Руководитель _____ ст. преподаватель В.С. Тарасов

Консультант _____ ассистент В.А. Ушаков

Воронеж 2026

СОДЕРЖАНИЕ

ОПРЕДЕЛЕНИЯ, ОБОЗНАЧЕНИЯ И СОКРАЩЕНИЯ	4
ВВЕДЕНИЕ.....	6
1 Обзор предметной области.....	9
1.1 Описание предметной области	9
1.2 Общее описание рынка образовательных IT - приложений	11
1.3 Обзор аналогов	13
1.3.1 Официальный сайт расписания cs.vsu.ru/rasp/.....	13
1.3.2 Телеграм-бот CSF Schedule.....	15
1.3.3 Google Sheets (мобильное приложение)	16
1.3.4 Результат сравнения аналогов.....	17
2 Проектирование системы	18
2.1 Пользовательские сценарии	18
2.2 Выбор стека технологий.....	21
2.3 Архитектура разрабатываемой системы.....	24
2.3.1 Серверная часть приложения.....	24
2.3.2 База данных	27
2.3.3 Клиентская часть приложения.....	28
2.3.4 Взаимодействие клиент-сервер	28
3 Реализация разрабатываемой системы	30
3.1 Серверная часть приложения.....	30
3.2 Клиентская часть приложения	31
3.2.1 Экран загрузки	32

3.2.2 Главный экран	32
3.2.3 Экран сохранения закладки расписания группы / преподавателя / аудитории	33
3.2.4 Экран сохранённого расписания	35
3.2.5 Боковое меню	36
3.2.6 Раздел источник данных.....	37
Заключение.....	39
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	40

ОПРЕДЕЛЕНИЯ, ОБОЗНАЧЕНИЯ И СОКРАЩЕНИЯ

В настоящей работе применяются следующие термины с соответствующими определениями:

Корутина — механизм языка Kotlin для написания асинхронного кода в последовательном стиле без блокировки потоков выполнения.

API — (Application Programming Interface) программный интерфейс приложения — набор правил и механизмов взаимодействия между программными компонентами.

XLSX — формат электронных таблиц Microsoft Excel на основе XML и ZIP-архива; используется как формат экспорта данных из Google Sheets.

HTTP — (HyperText Transfer Protocol) протокол передачи данных в сети, лежащий в основе обмена информацией в сети Интернет.

JSON — (JavaScript Object Notation) текстовый формат обмена данными, основанный на представлении объектов в виде пар «ключ — значение».

СУБД — система управления базами данных — программное обеспечение для создания, ведения и совместного использования баз данных.

ORM — (Object-Relational Mapping) объектно-реляционное отображение — технология, связывающая объекты программы с записями реляционной базы данных.

SQL — (Structured Query Language) язык структурированных запросов для работы с реляционными базами данных.

MVVM — (Model-View-ViewModel) архитектурный паттерн, разделяющий приложение на три слоя: модель данных, представление и модель представления.

UI — (User Interface) пользовательский интерфейс — совокупность средств взаимодействия пользователя с программой.

IDE — (Integrated Development Environment) интегрированная среда разработки программного обеспечения.

CRUD — (Create, Read, Update, Delete) базовый набор операций над данными: создание, чтение, обновление и удаление.

ER-диаграмма — (Entity-Relationship Diagram) диаграмма «сущность — связь», используемая для проектирования структуры базы данных.

SDK — (Software Development Kit) комплект средств разработки программного обеспечения для определённой платформы.

Room — библиотека абстракции над SQLite для платформы Android, обеспечивающая ORM-подход к работе с локальной базой данных.

SHA-256 — криптографическая хеш-функция, используемая для вычисления контрольной суммы загруженного файла и определения факта его изменения.

StateFlow — реактивный поток данных из библиотеки Kotlin Coroutines, используемый для хранения и распространения состояния пользовательского интерфейса.

ВВЕДЕНИЕ

Современное высшее образование характеризуется высокой динамичностью учебного процесса: расписание занятий регулярно обновляется, корректируются аудитории, преподаватели и состав групп. Для студентов и преподавателей факультета компьютерных наук Воронежского государственного университета актуальной задачей является оперативный доступ к актуальному расписанию занятий в удобном формате.

На сегодняшний день расписание факультета компьютерных наук размещается на официальном сайте факультета по адресу <https://www.cs.vsu.ru/rasp/> в виде ссылки на Google Sheets — облачную электронную таблицу [11]. Данный формат удобен для администраторов, однако для конечного пользователя, студента или преподавателя, он имеет ряд существенных недостатков: отсутствие нативного мобильного интерфейса, необходимость каждый раз открывать браузер, сложность быстрого поиска нужной группы или преподавателя среди множества строк таблицы, а также невозможность просмотра в офлайн-режиме без первичной загрузки данных.

Кроме того, расписание в табличном виде объединяет сразу несколько типов информации: расписание учебных групп, расписание преподавателей и занятость аудиторий. Пользователю, которому необходимо отслеживать только своё расписание (например, расписание одной группы или одного преподавателя), приходится вручную находить нужные строки и столбцы в громоздкой таблице. Это создаёт информационную перегрузку и снижает удобство использования.

Разрабатываемое в рамках настоящей курсовой работы мобильное приложение «CS Chedule» (Computer Science Schedule) представляет собой Android-платформу, предназначенную для просмотра расписания занятий факультета компьютерных наук ВГУ. Приложение автоматически загружает данные из Google Sheets при запуске, сохраняет их в локальную базу данных на

устройстве и предоставляет пользователю возможность сохранять «закладки» на интересующие расписания: группы, преподавателей или аудитории. После сохранения закладки пользователь может быстро перейти к просмотру расписания в структурированном виде — по дням недели, парам и типам недель (числитель/знаменатель).

Целью данной курсовой работы является проектирование и разработка мобильного приложения «CS Chedule», обеспечивающего удобный офлайн-доступ к расписанию занятий факультета компьютерных наук ВГУ с поддержкой автоматического импорта данных и персонализации через систему сохранённых расписаний.

Для достижения поставленной цели необходимо решить следующие задачи:

- провести анализ предметной области и изучить существующие аналоги сервисов просмотра расписания занятий;
- спроектировать архитектуру системы и схему локальной базы данных;
- выбрать технологический стек для реализации Android-приложения;
- реализовать модуль импорта данных из Google Sheets с парсингом формата XLSX;
- разработать клиентское Android-приложение с полным набором пользовательских экранов, включая главный экран с закладками, экран детального просмотра расписания и панель управления источником данных;
- провести тестирование корректности парсинга и отображения расписания.

Объектом исследования является процесс организации и предоставления учебного расписания в высшем учебном заведении. Предметом исследо-

вания выступают методы и инструменты программной реализации мобильного приложения для просмотра расписания занятий на платформе Android с локальным хранением данных.

Практическая значимость работы состоит в создании готового к использованию мобильного приложения, которое позволит студентам и преподавателям факультета компьютерных наук ВГУ быстро получать доступ к актуальному расписанию занятий, не прибегая к ручному поиску в объёмной электронной таблице, а также работать с расписанием в офлайн-режиме после первичной синхронизации данных.

1 Обзор предметной области

1.1 Описание предметной области

Предметной областью разрабатываемой системы является учебное расписание высшего учебного заведения — структурированное представление занятий, распределённых по семестрам, дням недели, временным слотам (парам), учебным группам, преподавателям и аудиториям. В отличие от простого списка событий, учебное расписание обладает сложной многомерной структурой, обусловленной спецификой организации учебного процесса в российских вузах.

Учебное расписание можно классифицировать по нескольким критериям:

- по объекту отображения: расписание групп, расписание преподавателей, занятость аудиторий;
- по временной периодичности: расписание на семестр, на учебную неделю, на конкретный день;
- по типу недели: числитель и знаменатель (чередующиеся варианты расписания);
- по способу представления: бумажное, электронное табличное (Excel, Google Sheets), веб-страница, мобильное приложение;
- по режиму доступа: онлайн (требует подключения к сети) и офлайн (данные хранятся локально).

Схема классификации учебных расписаний представлена на рисунке 1.

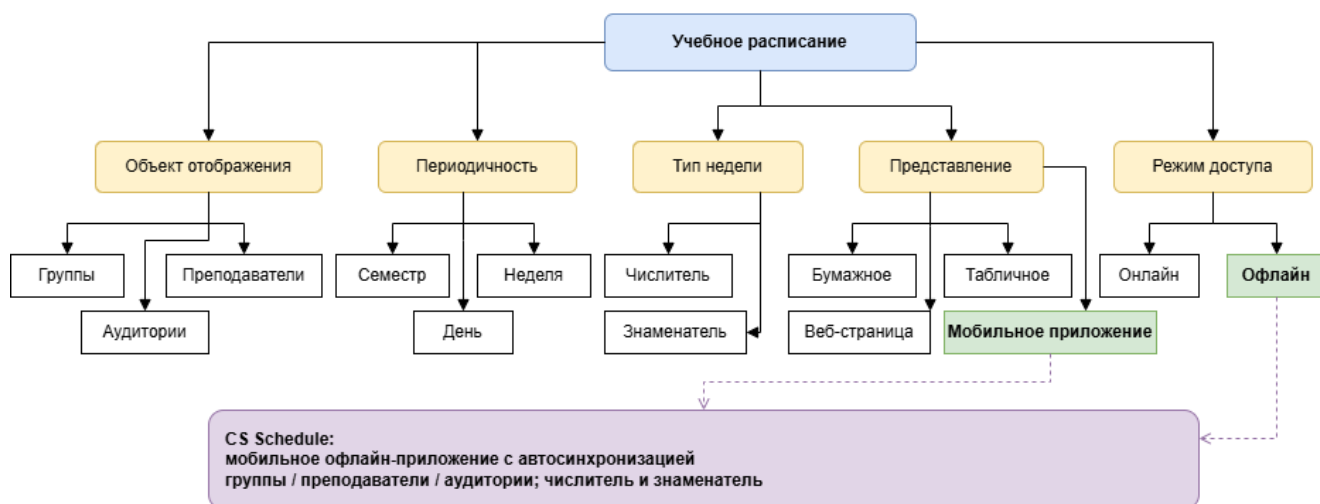


Рисунок 1 - Классификация учебных расписаний

Разрабатываемое приложение «CS Schedule» относится к категории мобильных офлайн-приложений с автоматической синхронизацией данных из облачного источника. Оно поддерживает все три типа объектов отображения (группы, преподаватели, аудитории), учитывает чередование числителя и знаменателя и предоставляет пользователю персонализированный доступ через систему сохранённых закладок.

Особенностью расписания факультета компьютерных наук ВГУ является его размещение в Google Sheets — облачной электронной таблице, доступ к которой осуществляется через ссылку на официальном сайте кафедры. Таблица содержит несколько листов, каждый из которых соответствует определённому семестру. Такой подход к публикации расписания широко распространён в вузах, поскольку позволяет администраторам оперативно вносить изменения без привлечения разработчиков, однако создаёт определённые трудности при автоматизированной обработке данных на стороне клиентского приложения.

Структура листа включает: строки с номерами курсов и групп, блоки дней недели (понедельник — суббота), временные слоты пар в колонке В, а также ячейки с информацией о занятиях (название предмета, преподаватель, аудитория). Ячейки могут быть объединены (merged cells), что отражает под-

групповое деление внутри одной группы. Наличие объединённых ячеек является одной из ключевых особенностей формата, требующей специальной обработки при парсинге.

Основные особенности формата исходных данных:

- Объединённые ячейки. Значение объединённой ячейки распространяется на все входящие в неё позиции.
- Блочная структура по дням недели. Каждая пара представлена двумя строками: числитель и знаменатель.
- Формат ячейки занятия: название дисциплины, преподаватель, аудитория.
- Дистанционные занятия маркируются кодом «ДО» и исключаются из справочника аудиторий.
- Типичный файл содержит более 5500 занятий, 120+ групп, 150+ преподавателей и 30+ аудиторий.

Таким образом, разработка мобильного приложения с автоматическим импортом, локальным хранением и персонализированным интерфейсом отвечает на запросы студентов и преподавателей, которым необходим быстрый и удобный доступ к актуальному расписанию непосредственно с мобильного устройства.

1.2 Общее описание рынка образовательных IT - приложений

Рынок образовательных IT-приложений в настоящее время представляет собой динамично развивающийся сегмент программного обеспечения, охватывающий широкий спектр решений для различных участников образовательного процесса. К основным категориям таких приложений относятся: платформы массового открытого онлайн-обучения (МООС), электронные днев-

ники и журналы, системы дистанционного обучения (LMS), мобильные организаторы для студентов, а также специализированные сервисы просмотра расписания занятий.

Согласно общемировым тенденциям цифровизации образования, всё больше вузов переходят от бумажных и статичных электронных форматов расписания к облачным решениям. Наиболее распространёнными являются публикация расписания в виде PDF-документов, HTML-страниц или облачных таблиц (Google Sheets, Microsoft Excel Online). При этом мобильные приложения, ориентированные непосредственно на просмотр расписания конкретного учебного заведения, занимают относительно узкую нишу, но при этом обладают высокой востребованностью среди целевой аудитории.

С точки зрения пользовательского опыта, современные студенты и преподаватели предъявляют к образовательным приложениям следующие ожидания: быстрый доступ к информации, адаптивный интерфейс под мобильные устройства, возможность работы без постоянного подключения к сети, персонализация отображаемого контента. Универсальные решения, такие как Google Sheets или веб-браузеры, формально удовлетворяют потребность в доступе к данным, однако не обеспечивают оптимального пользовательского опыта при ежедневном использовании.

Следует отметить, что большинство существующих решений для просмотра расписания в российских вузах либо представляют собой веб-страницы с табличным представлением, не адаптированные под мобильные устройства, либо являются универсальными приложениями-агрегаторами, не интегрированными с конкретным источником данных факультета. Ниша специализированных мобильных приложений с автоматическим импортом расписания из официальных источников конкретного факультета заполнена недостаточно, что создаёт предпосылки для разработки целевого решения.

Разрабатываемое приложение «CS Schedule» позиционируется как специализированный мобильный клиент для факультета компьютерных наук ВГУ,

сочетающий автоматический импорт данных из официального источника, локальное хранение для офлайн-доступа и персонализированный интерфейс с системой закладок. Такой подход соответствует современным требованиям к образовательным мобильным приложениям и заполняет выявленный пробел на рынке.

1.3 Обзор аналогов

Для обоснования необходимости разработки собственного решения был проведён сравнительный анализ существующих сервисов и приложений, которые потенциально могут использоваться для просмотра расписания занятий факультета компьютерных наук ВГУ. В качестве аналогов рассмотрены официальный сайт факультета, телеграм-бот CSF Schedule и мобильное приложение Google Sheets.

Для выявления сильных и слабых сторон существующих решений были определены общие критерии сравнения:

- поддержка просмотра расписания групп, преподавателей и аудиторий;
- удобство использования
- возможность работы в офлайн-режиме;
- наличие системы персонализации (сохранение избранных расписаний);

1.3.1 Официальный сайт расписания [cs.vsu.ru/rasp/](https://www.cs.vsu.ru/rasp/)

Официальный сайт расписания факультета компьютерных наук ВГУ по адресу <https://www.cs.vsu.ru/rasp/> [1] является первичным и наиболее авторитетным источником. На странице размещена ссылка на Google Sheets с актуальным расписанием текущего семестра. Сайт предоставляет доступ к полным

данным расписания и является единственным официальным каналом публикации информации об учебном процессе факультета.

Преимуществами данного решения являются актуальность данных, отсутствие необходимости установки дополнительного программного обеспечения и доступность с любого устройства, имеющего веб-браузер. Вместе с тем, сайт не имеет специализированного мобильного интерфейса: при открытии на смартфоне пользователь перенаправляется на облачную таблицу, не адаптированную для удобного просмотра на малом экране. Офлайн-доступ отсутствует, персонализация не предусмотрена, а поиск нужной группы или преподавателя требует ручного пролистывания таблицы.

Иллюстрация страницы расписания представлена на рисунке 2.

	A	B	CB	CC	CD
1			3 курс		
2			11 группа	12 группа	13 группа
3		Часы звонков	Направление "ИБ"		Специальность
4			"Безопасность компьютерных систем"		специализация "АБКС"
22	Вторник	8:00 - 9:35			Экспериментально-исследовательская практика
23					
24		9:45 - 11:20	ая Н.П. 383	Ин.яз. доц. Барабушка И.А. 309П	Экспериментально-исследовательская практика
25				ЗИ от утечки по ТК (id=29482) асс. Канюс А.С. 303П	
26		11:30-13:05	Безопасность операционных систем (id=4418) доц. Самодуров А.С. 301П	КТ доц. Стадная Н.П. 297	
27			Безопасность операционных систем (id=4418) доц. Самодуров А.С. 387	Ин.яз. доц. Барабушка И.А. 309П	
28		13:25-15:00	ЗИ от утечки по ТК (id=29482) асс. Канюс А.С. 303П	СЦВЗ (id=3420) доц. Митрофанова Е.Ю. 290	Физическая культура и спорт
29					
30		15:10-16:45	Митрофанова Е.Ю. 290	ЗИ от утечки по ТК (id=29482) асс. Канюс А.С. 303П	СЦВЗ (id=3420) доц. Митрофанова Е.Ю. 290
31					
32		16:55-18:30	СиСПИ (id=4035) доц. Самодуров А.С. 295		ЗИ от утечки по ТК (id=29482) проф. Головинский П.А. 303П
33					
34		18:40-20:00			
35					
36		20:10-21:30			
37					
38					
39		8:00 - 9:35	рия (id=9853) ст.преп. Вяткина А.Г. (ДО)		
40					

Рисунок 2 - Расписание с официального сайта ФКН

1.3.2 Телеграм-бот CSF Schedule

CSF Schedule — недавно появившийся бот в Telegram для просмотра расписания факультета компьютерных наук. Бот предлагает множество удобных функций: просмотр расписания студента (при регистрации необходимо указать группу, подгруппу и курс), однако возможно посмотреть расписание только для своей группы, чтобы посмотреть чужое — необходимо в настройках изменить свои данные, после чего обновится расписание. Есть расписание сессии, поиск свободной аудитории, но всё это можно сделать только онлайн — никакого оффлайн-формата. А самая главная проблема — на данный момент в Российской Федерации Telegram замедлен.

Иллюстрация главной страницы бота представлена на рисунке 3.

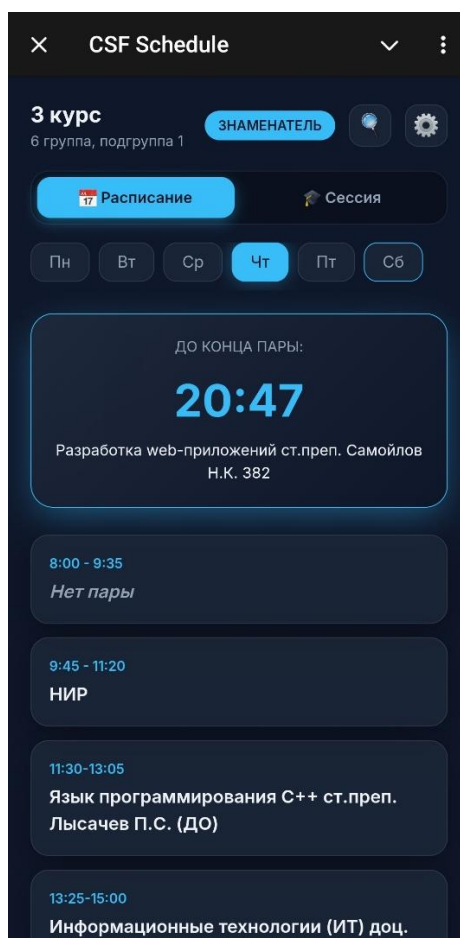


Рисунок 3 - Мини-приложение CSF Schedule

1.3.3 Google Sheets (мобильное приложение)

Google Sheets — официальное мобильное приложение Google для работы с облачными электронными таблицами. Поскольку расписание факультета компьютерных наук размещено именно в Google Sheets, теоретически пользователь может открыть таблицу непосредственно через это приложение, получив доступ к полным данным расписания.

Однако Google Sheets является универсальным инструментом для работы с таблицами, а не специализированным просмотрщиком расписания. Приложение не предоставляет структурированного отображения по дням недели и парам, не поддерживает систему закладок на конкретные группы или преподавателей, не учитывает семантику числителя и знаменателя. Просмотр расписания в Google Sheets на мобильном устройстве сопровождается необходимостью горизонтальной и вертикальной прокрутки объёмной таблицы, что существенно снижает удобство использования.

Иллюстрация отображения расписания в Google Sheets на мобильном устройстве представлена на рисунке 4.

	A	B	C	D
1				1 курс
2				
3		Часы звонков		1 группа
4				
5		8:00 - 9:35		
6				
7		9:45 - 11:20	История России доц. Лавлинский С.А. 292	
8				
9		11:30-13:00	Дискретная математика доц. Попов М.И. 477	
10		5		
11		13:25-15:00		
12		0		
13		15:10-16:45		
14		5		
15		16:55-18:30	Иняз. преп. Семенов В.П. 309П	
16		0		
17		18:40-20:00		
18		0		
19		20:10-21:30		
20		0		

Рисунок 4 - Просмотр расписания в приложении Google Sheets

1.3.4 Результат сравнения аналогов

Результаты сравнения функциональных возможностей аналогов представлены в таблице 1.

Средство просмотра расписания	Группы / Преподаватели / Аудитории	Удобство использования	Оффлайн режим	Закладки
Официальный сайт	—	—	—	—
CSF Schedule	+	+	—	—
Google Sheets (мобильное приложение)	—	—	+	—

Таблица 1 – Сравнение функциональных возможностей аналогов

Анализ показал, что ни один из рассмотренных аналогов не сочетает поддержку всех трёх типов расписания, полноценный офлайн-режим и систему персонализированных закладок в едином решении. Это подтверждает актуальность разработки специализированного приложения «CS Schedule».

2 Проектирование системы

2.1 Пользовательские сценарии

Для определения основных функций мобильного приложения были сформированы пользовательские сценарии с помощью диаграммы прецедентов (use case diagram). Данная диаграмма наглядно отображает взаимодействие актора (пользователя) с функциональными возможностями системы.

Спроектированная диаграмма прецедентов представлена на рисунке 5.

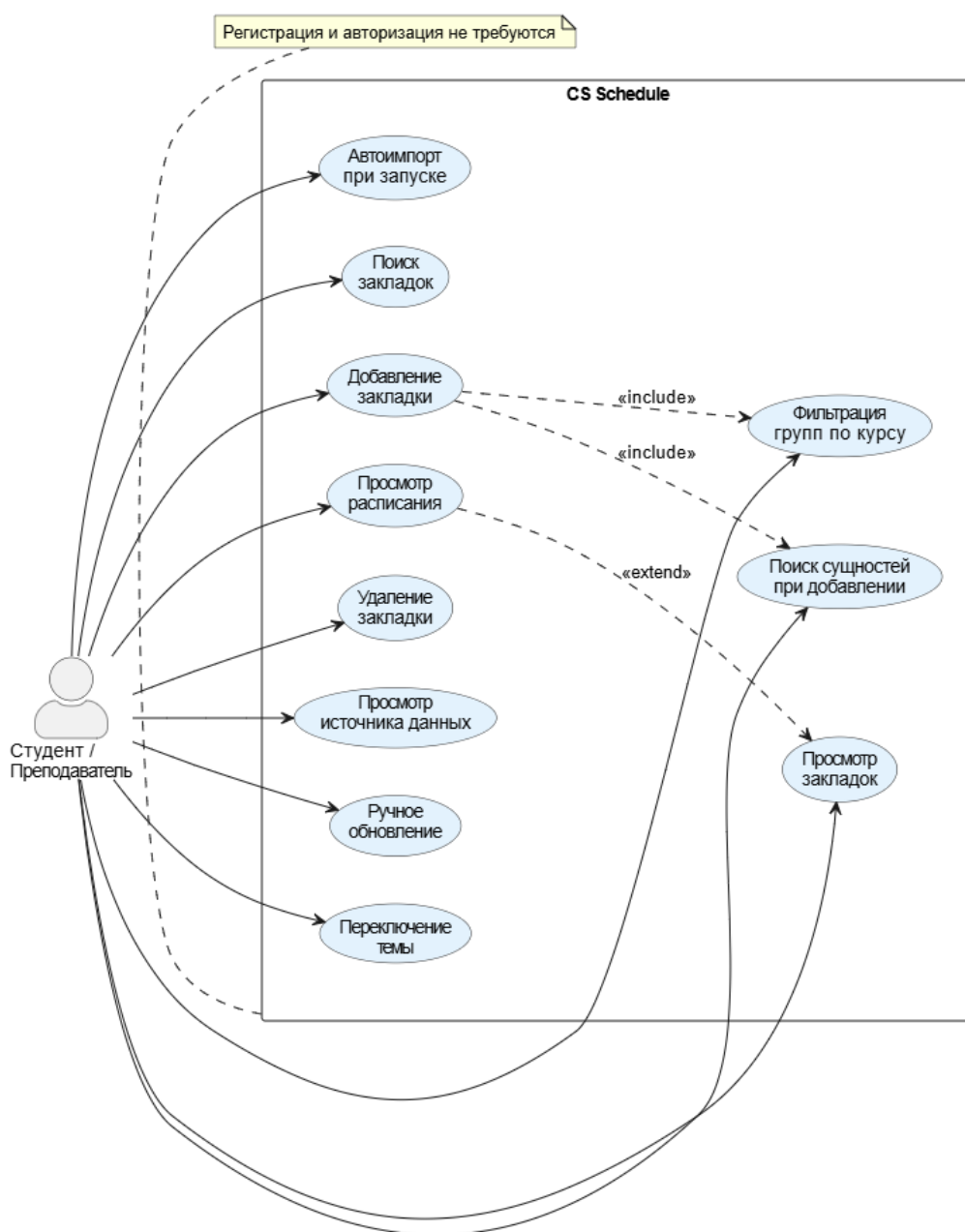


Рисунок 5 - Диаграмма прецедентов

В системе предусмотрен один тип пользователя — «Студент / Преподаватель» (конечный пользователь приложения). Регистрация и авторизация не требуются: приложение работает локально без учётной записи, что упрощает начало использования и снижает порог входа для новых пользователей.

Основные пользовательские сценарии приложения:

- запуск приложения с автоматическим импортом расписания;
- просмотр списка сохранённых расписаний на главном экране;
- поиск среди сохранённых расписаний по названию;
- добавление новой закладки: выбор типа (группа / преподаватель / аудитория) → поиск и выбор сущности → сохранение;
- при выборе группы: фильтрация по курсу через панель вкладок;
- просмотр детального расписания выбранной закладки (дни → пары → числитель/знаменатель);
- удаление сохранённого расписания;
- просмотр информации об источнике данных через боковое меню;
- ручной запуск обновления данных;
- переключение тёмной темы через боковое меню.

Для детализации наиболее значимых сценариев взаимодействия пользователя с системой были разработаны диаграммы последовательности (sequence diagrams). Данные диаграммы показывают порядок обмена сообщениями между компонентами системы при выполнении конкретного сценария.

Диаграмма последовательности для сценария «Добавление закладки на расписание группы» представлена на рисунке 6. На диаграмме отражены взаимодействия между пользовательским интерфейсом, ViewModel, репозиторием и базой данных при сохранении новой закладки.

Рисунок 6 - Диаграмма последовательности: добавление закладки

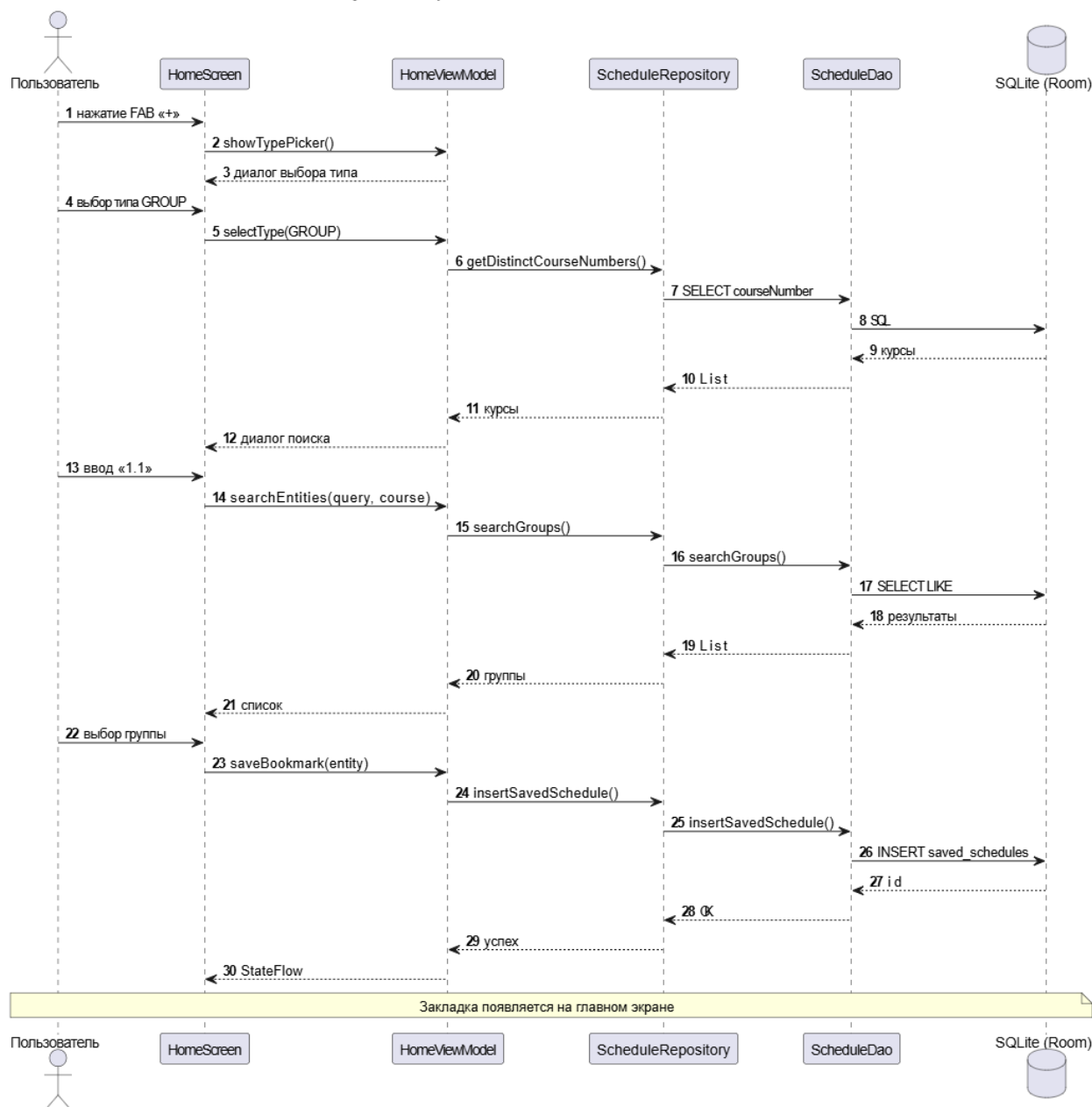


Рисунок 6 - Диаграмма последовательности: добавление закладки

Диаграмма последовательности для сценария «Автоматический импорт при запуске» представлена на рисунке 7. Данный сценарий является ключевым для обеспечения актуальности данных и демонстрирует полный цикл: от HTTP-запроса к сайту факультета до записи распарсенных данных в локальную базу.

Рисунок 7 - Диаграмма последовательности: автоматический импорт

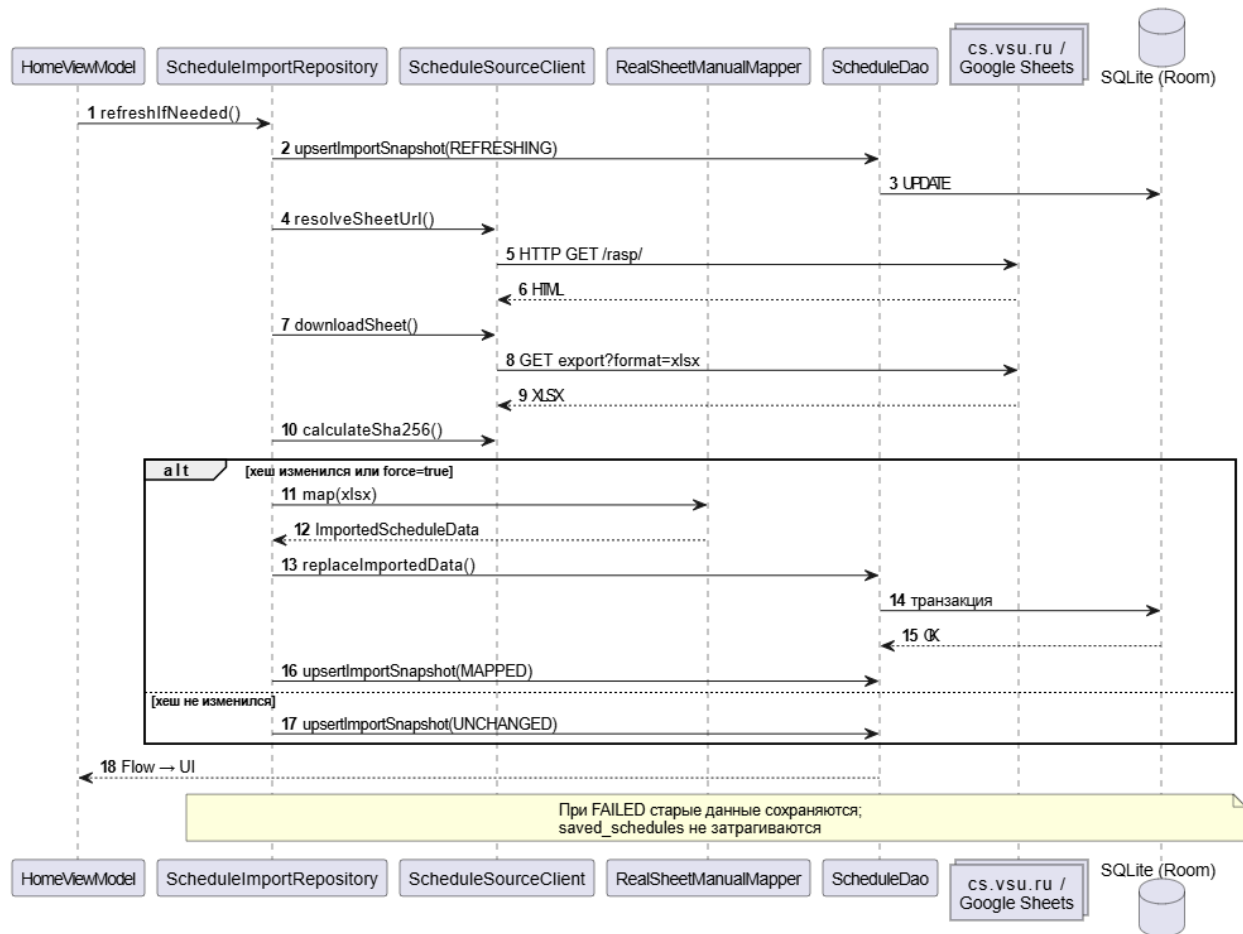


Рисунок 7 - Диаграмма последовательности: автоматический импорт

2.2 Выбор стека технологий

Определение технологического стека — важный этап проектирования, от которого зависят функциональные возможности, скорость работы и удобство дальнейшей поддержки системы. При подборе инструментов ориентировались на официальные рекомендации Google для Android, доступность документации и активное сообщество разработчиков, возможность обойтись без выделенного сервера, а также на практический опыт реализации подобных задач.

Развёртывание отдельного серверного приложения не предполагается: загрузка, преобразование и сохранение расписания выполняются непосредственно на смартфоне пользователя. Такой подход облегчает ввод системы в

эксплуатацию и исключает затраты на серверную инфраструктуру, однако требует внимательной оптимизации импорта и учёта ограничений мобильного устройства по объёму обрабатываемых данных.

В качестве языка разработки выбран Kotlin (версия 2.0.21) — статически типизированный язык для JVM, который Google позиционирует как основной для Android [2]. Kotlin отличается компактным синтаксисом, встроенной защитой от ошибок с null-ссылками, встроенной поддержкой корутин и полной взаимозаменяемостью с Java-кодом. Эти свойства, а также развитая экосистема библиотек, делают его оптимальным выбором для клиентской части приложения.

Пользовательский интерфейс строится на Jetpack Compose (BOM 2024.09.00) — декларативном фреймворке Google для Android [3]. Интерфейс описывается Kotlin-функциями с аннотацией `@Composable`, оформление соответствует Material Design 3 [12], а при изменении состояния экран перерисовывается автоматически. По сравнению с классическими XML-макетами Compose уменьшает количество повторяющегося кода и облегчает реализацию динамических экранов.

Структура приложения основана на паттерне MVVM (Model–View–ViewModel) [4]: представление отделено от бизнес-логики, а актуальное состояние экрана хранится во ViewModel и передаётся в UI через StateFlow. Google рекомендует этот подход как базовый для Android; он сохраняет данные при смене ориентации экрана и переключении темы.

Переходы между экранами реализованы с помощью Navigation Compose (версия 2.9.0) — библиотеки типобезопасной навигации в рамках паттерна Single Activity [6]. Одна Activity выступает контейнером для всего приложения, что упрощает контроль жизненного цикла и соответствует актуальной архитектурной модели Android.

Локальное хранение организовано через Room (версия 2.7.2) — обёртку Google над SQLite с ORM-подходом [5]. Таблицы описываются классами

Entity, запросы — интерфейсами Dao, конфигурация — классом Database; вспомогательный код генерирует KSP. Room проверяет SQL на этапе компиляции и позволяет подписываться на изменения данных через Flow.

Настройки приложения сохраняются в SharedPreferences — хранилище пар «ключ — значение». В «CS Schedule» через него запоминается выбранная пользователем тёмная или светлая тема.

Сетевое взаимодействие реализовано на OkHttp (версия 4.12.0) [8]: с его помощью загружается HTML-страница кафедры и скачивается XLSX-экспорт из Google Sheets. Библиотека устойчиво обрабатывает сетевые запросы, следует перенаправлениям и позволяет задавать таймауты.

Для разбора HTML-страницы применяется Jsoup (версия 1.18.1) [9]. Она используется, чтобы найти на странице cs.vsu.ru/rasp/ ссылку на таблицу Google Sheets; API библиотеки удобен для обхода DOM-дерева документа.

Парсинг XLSX выполняет собственный компонент RealSheetManualMapper без подключения Apache POI [10]. Файл рассматривается как ZIP-архив, внутренние XML читаются через XmlPullParser. Отказ от тяжёлой внешней библиотеки сокращает размер APK и даёт полный контроль над разбором нестандартного формата расписания факультета компьютерных наук.

Сборка проекта автоматизирована Android Gradle Plugin (версия 8.13.2): плагин управляет зависимостями, компиляцией исходного кода и формированием APK.

Генерация служебного кода Room выполняется процессором KSP (версия 2.0.21-1.0.25) — инструментом обработки аннотаций Kotlin, работающим на этапе компиляции.

Модульные тесты написаны на JUnit 4 с привлечением Robolectric, который эмулирует Android-среду на JVM [13]. Это позволяет проверять парсер XLSX без запуска эмулятора и ускоряет итерации при разработке.

2.3 Архитектура разрабатываемой системы

Архитектура задаёт, из каких модулей состоит программа, как они связаны между собой и по каким правилам обмениваются данными. Грамотно выстроенная структура повышает модульность кода, упрощает добавление новых функций и облегчает сопровождение приложения.

«CS Schedule» реализован как одноклиентное приложение с локальной базой данных: центральный сервер не используется, загрузка и обработка расписания происходят на телефоне, а обмен с внешним миром ограничивается HTTP-запросами к сайту факультета и Google Sheets.

Схема компонентов и их связей приведена на рисунке 8.

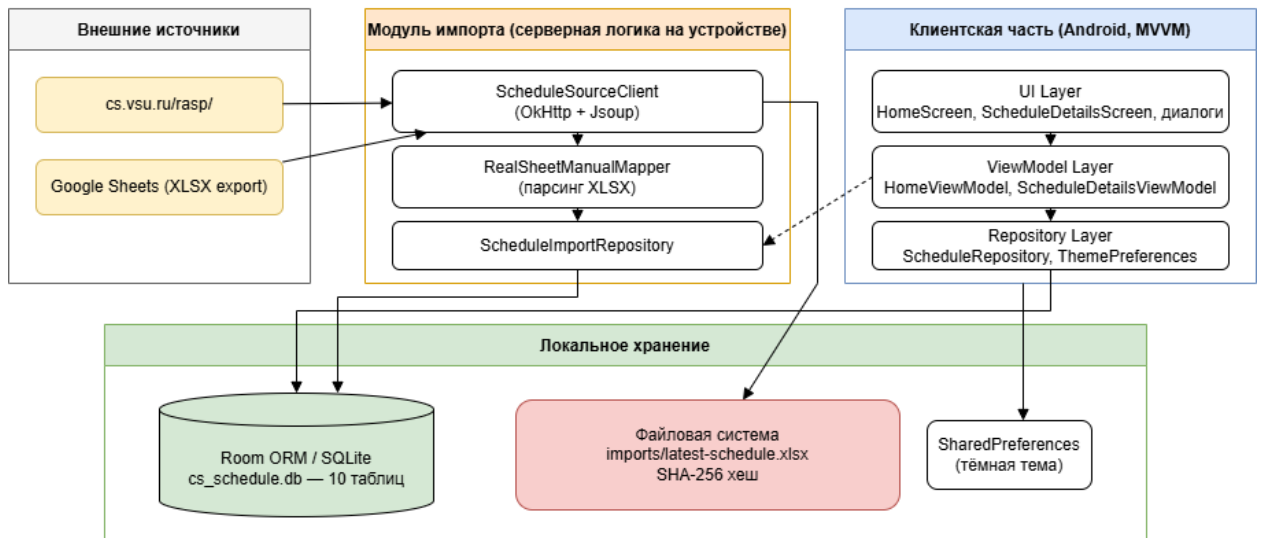


Рисунок 8 - Общая архитектура системы «CS Schedule»

2.3.1 Серверная часть приложения

В рамках выбранной архитектуры под «серверной частью» понимается не отдельная машина, а программный модуль импорта на устройстве пользователя. Он выполняет те же задачи, что и `backend` в классической схеме «кли-

ент — сервер»: получает расписание из внешнего источника, проверяет, изменились ли данные, и превращает таблицу Excel в набор записей реляционной БД. Модуль импорта состоит из трёх классов:

- `ScheduleSourceClient` — обращение к `cs.vsu.ru/rasp/`, поиск ссылки на Google Sheets через Jsoup, загрузка XLSX средствами OkHttp, расчёт SHA-256;
- `RealSheetManualMapper` — разбор XLSX с учётом объединённых ячеек и особенностей таблицы ФКН;
- `ScheduleImportRepository` — координация всего цикла импорта, ведение статусов и вызов `ScheduleDao.replaceImportedData()` для записи в базу.

Последовательность шагов импорта показана на рисунке 9.

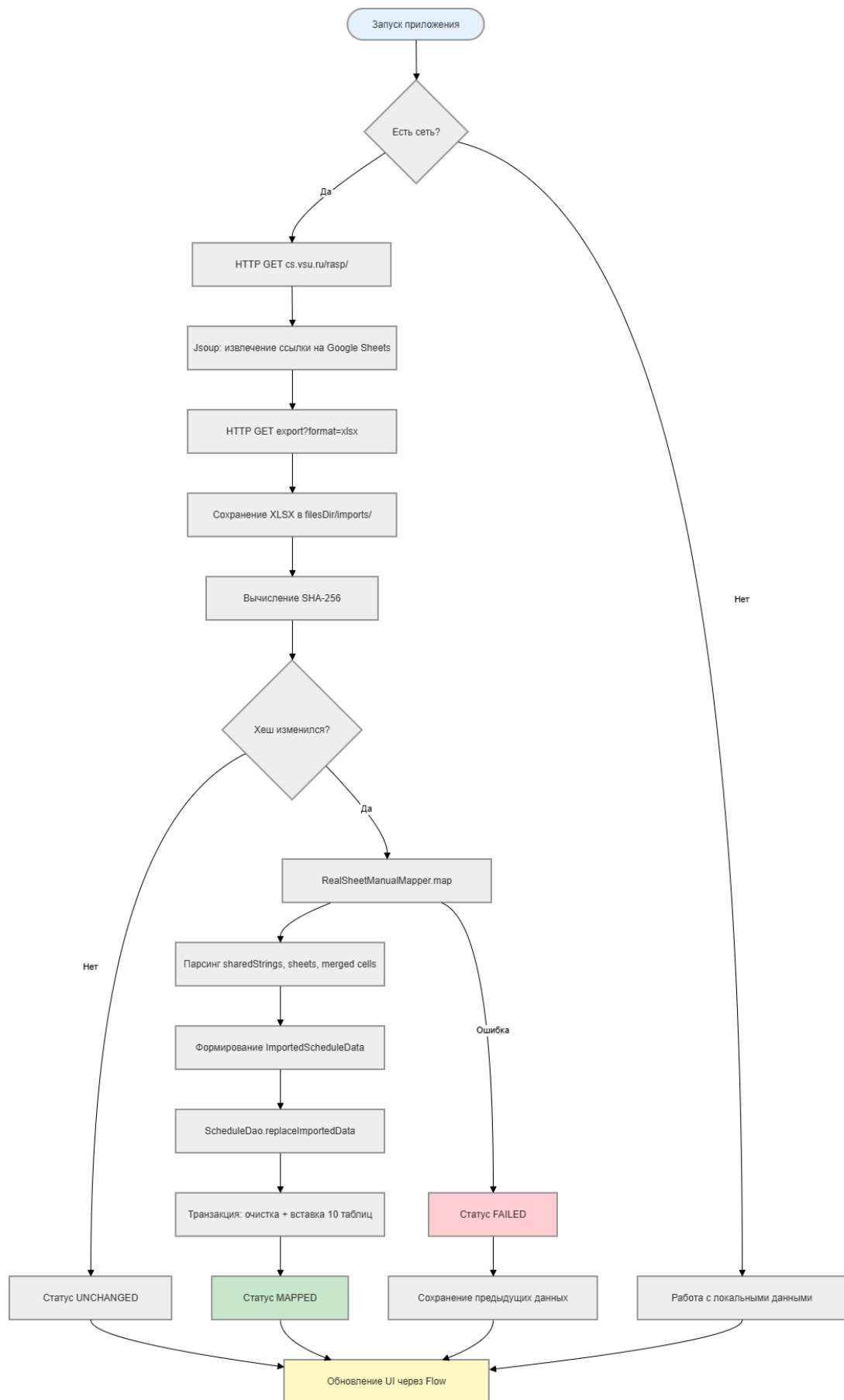


Рисунок 9 - Цепочка импорта данных

2.3.2 База данных

Постоянное хранение обеспечивает SQLite, доступ к которому организован через Room ORM. SQLite входит в состав Android, быстро обрабатывает локальные запросы и не требует интернета для чтения уже загруженного расписания.

Файл базы данных cs_schedule.db содержит 10 таблиц; их взаимосвязи отражены на ER-диаграмме (рисунок 10).

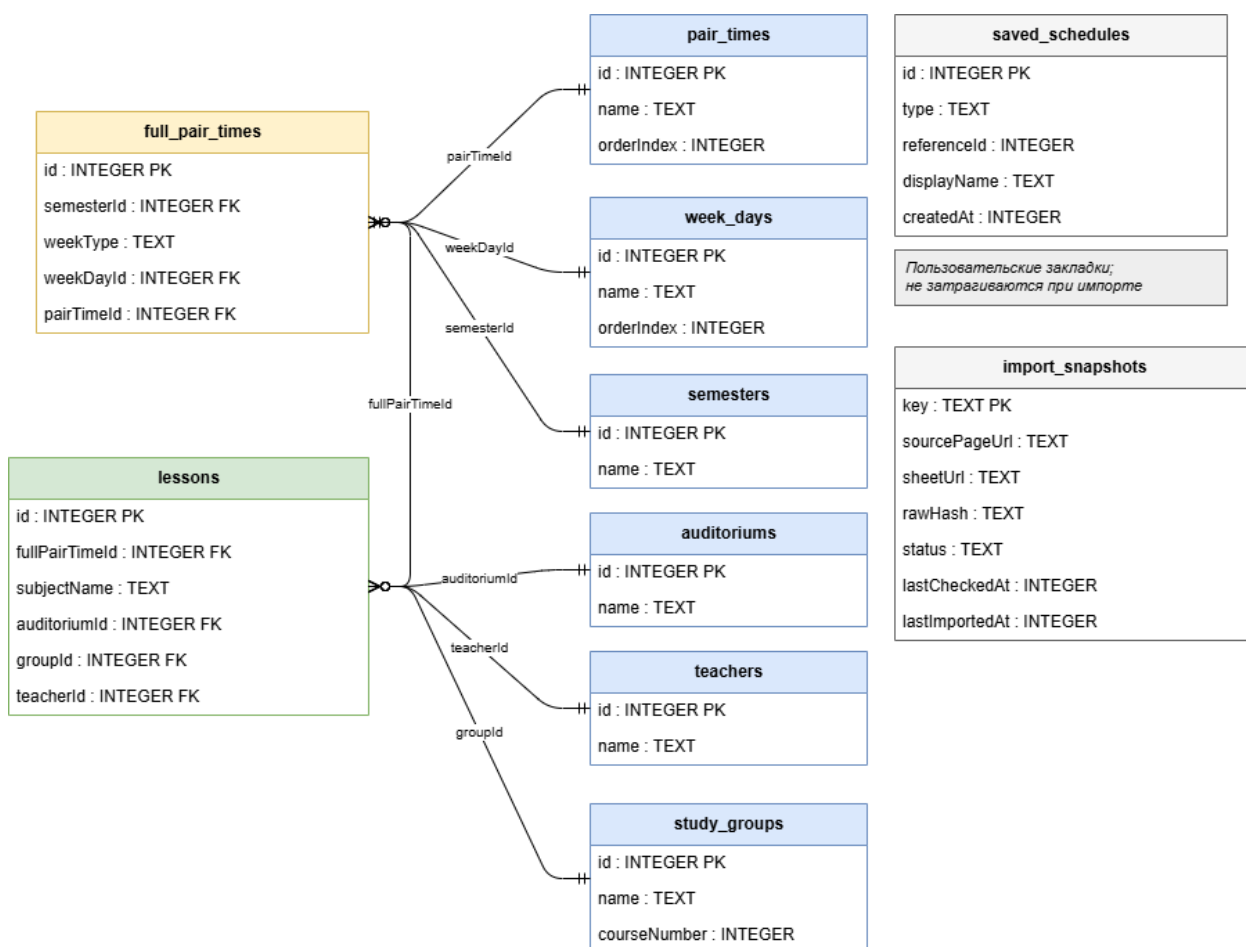


Рисунок 10 - ER-диаграмма базы данных

В схему входят таблицы: semesters, week_days, pair_times, full_pair_times, teachers, auditoriums, study_groups, lessons, saved_schedules, import_snapshots.

Семестры, дни недели, пары, преподаватели, аудитории, группы и занятия заполняются при импорте и перезаписываются при каждом обновлении.

Таблица `saved_schedules` хранит закладки пользователя и при импорте не очищается. В `import_snapshots` сохраняются служебные сведения о последней синхронизации: хеш файла, метки времени, текущий статус операции.

Обновление импортированных данных выполняет метод `replaceImportedData()` внутри одной транзакции: сначала очищаются затронутые таблицы, затем в них пакетно вставляются новые строки. Если импорт прервётся, пользователь не увидит «полуготовое» расписание — либо останутся старые данные, либо запишется полный новый набор.

2.3.3 Клиентская часть приложения

Интерфейс и прикладная логика оформлены как нативное Android-приложение на Kotlin с Jetpack Compose по схеме MVVM. Код разбит на четыре слоя:

- UI Layer: `HomeScreen`, `ScheduleDetailsScreen`, диалоги выбора, боковое меню;
- `viewModel` Layer: `HomeViewModel`, `ScheduleDetailsViewModel`, `StateFlow`;
- Repository Layer: `ScheduleRepository`, `ScheduleImportRepository`, `ThemePreferencesRepository`;
- Data Layer: `ScheduleDao`, `ScheduleSourceClient`, `RealSheetManualMapper`, `SharedPreferences`.

Разделение обязанностей следующее: UI только рисует экран и принимает ввод, `ViewModel` держит состояние, `Repository` объединяет несколько источников данных, нижний слой работает с БД, сетью и файлами.

Маршрутизация экранов — через `Navigation Compose`; `MainActivity` содержит `NavHost` с путями `home` и `details/{type}/{referenceId}/{title}`.

2.3.4 Взаимодействие клиент-сервер

Обращения к внешним ресурсам инициируют и клиентский модуль, и блок импорта по протоколу HTTP. Запросы уходят в фоновые корутины Kotlin [7], поэтому интерфейс не «зависает» во время загрузки.

Используются два адреса:

- <https://www.cs.vsu.ru/rasp/> — страница с HTML и ссылкой на таблицу;
- <https://docs.google.com/spreadsheets/d/{id}/export?format=xlsx> — прямая выгрузка XLSX.

HTTP-клиент — OkHttp 4.12.0. Если сеть недоступна, просмотр закладок продолжается по локальной копии расписания; в разделе «Источник данных» отображается соответствующее состояние.

3 Реализация разрабатываемой системы

3.1 Серверная часть приложения

Код импорта сосредоточен в пакете `ru.vsu.csschedule.data.importing`. Именно здесь сосредоточена основная сложность проекта: нужно корректно прочитать объёмную таблицу Excel и разложить её по нормализованным таблицам SQLite. В модуле три ключевых класса, каждый закрывает свой этап цепочки.

`ScheduleSourceClient` инкапсулирует всё сетевое взаимодействие с внешними источниками. Остальные компоненты не знают деталей HTTP и HTML — они обращаются к этому классу за ссылкой на таблицу и за байтами XLSX-файла.

Метод `resolveSheetUrl(sourcePageUrl)` отправляет GET-запрос на `https://www.cs.vsu.ru/rasp/`, передаёт ответ в Jsoup и извлекает URL Google Sheets. Метод `downloadSheet(sheetUrl)` собирает адрес экспорта в формате XLSX и загружает файл через OkHttp. Метод `calculateSha256(bytes)` считает SHA-256 от содержимого, чтобы понять, менялась ли таблица с прошлого раза. Метод `persistDownloadedSheet(context, bytes)` кладёт копию файла в `context.filesDir/imports/latest-schedule.xlsx` — при необходимости парсинг можно повторить без повторной загрузки из сети.

`ScheduleImportRepository` — «дирижёр» импорта: он по очереди вызывает `ScheduleSourceClient`, `RealSheetManualMapper` и `ScheduleDao` и переводит процесс из состояния в состояние. Через него же UI запускает обновление расписания.

Алгоритм метода `refreshIfNeeded(force: Boolean)`:

1. В `import_snapshots` выставляется статус REFRESHING.
2. Через `ScheduleSourceClient.resolveSheetUrl()` получается актуальная ссылка на таблицу.
3. `ScheduleSourceClient.downloadSheet()` скачивает XLSX.

4. Считается SHA-256 и сравнивается с сохранённым rawHash; если хеши совпали и force=false, разбор файла пропускается.

5. Иначе RealSheetManualMapper.map() превращает XLSX в объект ImportedScheduleData.

6. При успехе вызывается ScheduleDao.replaceImportedData(), статус меняется на MAPPED.

7. При сбое — статус FAILED, но ранее загруженные строки в БД остаются, и пользователь по-прежнему видит старое расписание.

Такое поведение при ошибках сети или парсинга сохраняет работоспособность приложения: офлайн-просмотр не прерывается из-за временного сбоя синхронизации.

RealSheetManualMapper содержит самописный парсер XLSX, заточенный под реальный файл расписания ФКН. От Apache POI сознательно отказались — библиотека раздула бы APK более чем на 10 МБ.

Вложенный XlsxWorkbookParser открывает архив ZipFile и читает sharedStrings.xml, workbook.xml и worksheets/sheetN.xml. Отдельно обрабатываются merged cells: текст из объединённой ячейки подставляется во все координаты входящего в неё диапазона.

3.2 Клиентская часть приложения

Пользовательская часть написана на Kotlin с Jetpack Compose по схеме MVVM. При старте CSScheduleApplication поднимает Room, OkHttp, репозитории и mapper; MainActivity открывает AppNavGraph — дерево переходов между экранами. Вход по логину и паролю не предусмотрен: все данные привязаны к устройству, а не к учётной записи на сервере.

Исходники сгруппированы в модуле :app: ru.vsu.csschedule.ui.home и ui.details — интерфейс, data.importing — импорт, data.local — сущности и DAO, data.repository — репозитории.

3.2.1 Экран загрузки



Рисунок 11 - Экран загрузки

3.2.2 Главный экран

HomeScreen — основная рабочая область, откуда доступны все ключевые действия.

На экране размещены:

- ModalNavigationDrawer (боковое меню) с разделом «Источник данных» и переключателем тёмной темы;
- OutlinedTextField для фильтрации списка закладок по названию;
- LazyColumn с сохранёнными расписаниями;
- FloatingActionButton «+» для добавления новой закладки;

Нажатие на «+» открывает выбор типа (группа, преподаватель, аудитория), затем — окно поиска с фильтром по курсу для групп. Состоянием экрана управляет HomeViewModel через StateFlow, поэтому список и статусы обновляются без ручного обновления UI.

Внешний вид главного экрана показан на рисунке 12.

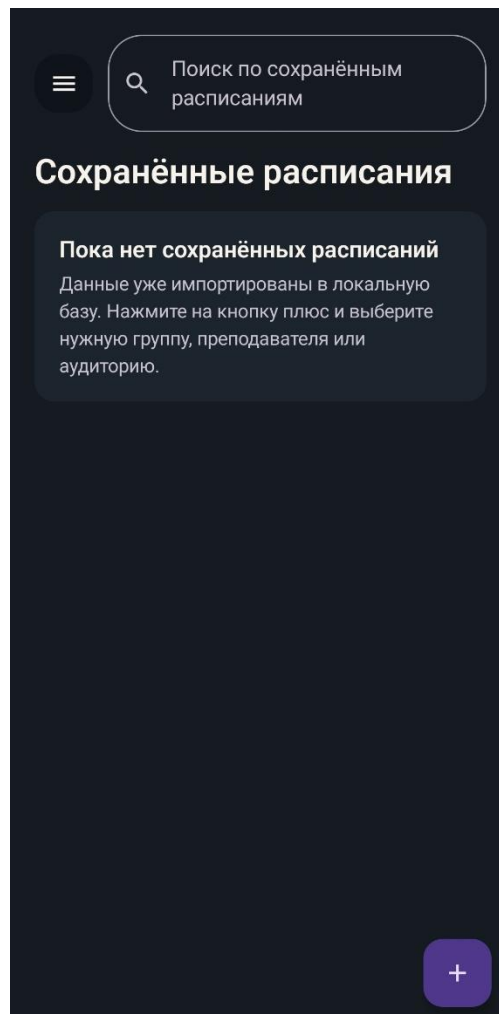


Рисунок 12 - Главный экран приложения (HomeScreen)

3.2.3 Экран сохранения закладки расписания группы / преподавателя / аудитории

Выбор группы, преподавателя или аудитории для новой закладки выполняется не на отдельной странице-каталоге, а во всплывающем диалоге. Для

типа «Группа» диалог содержит строку поиска, вкладки курсов и прокручиваемый перечень групп, сужаемый по мере ввода текста.

В зависимости от контекста используется `ModalBottomSheet` или `AlertDialog`; оба варианта позволяют быстро найти нужную позицию среди более чем 120 групп. Для преподавателей и аудиторий предусмотрены такие же диалоги с соответствующими списками.

Пример диалога выбора группы приведён на рисунке 13.

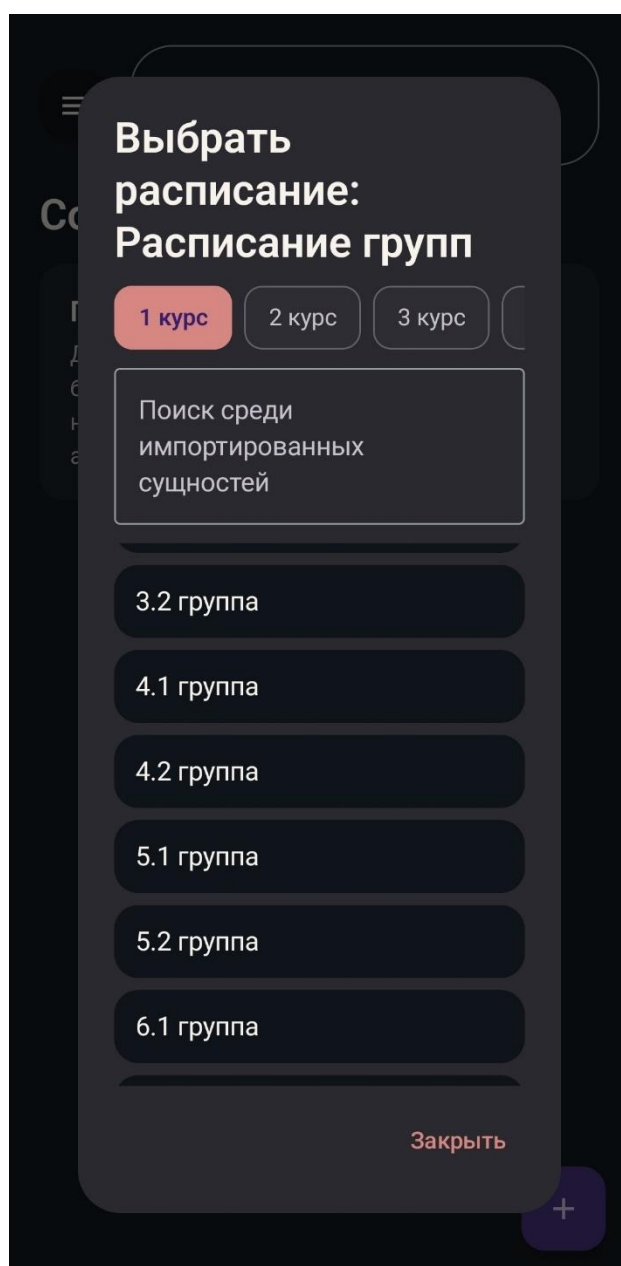


Рисунок 13 - Диалог выбора расписания группы

3.2.4 Экран сохранённого расписания

Подробное расписание открывается по нажатию на закладку и реализовано в `ScheduleDetailsScreen`. Адрес в навигации: `details/{type}/{referenceId}/{title}`.

Структура экрана:

- `CenterAlignedTopAppBar` с именем выбранной группы, преподавателя или аудитории;
- `LazyColumn`, разбитый по дням с понедельника по субботу;
- внутри каждого дня — пары с подразделами «Числитель» и «Знаменатель» (предмет, преподаватель, аудитория).

`ScheduleDetailsViewModel` по аргументам `type` и `referenceId` запрашивает строки у `ScheduleRepository` и собирает из них удобное для отображения дерево «день → пара → занятие».

Экран детального просмотра иллюстрируется рисунком 14.

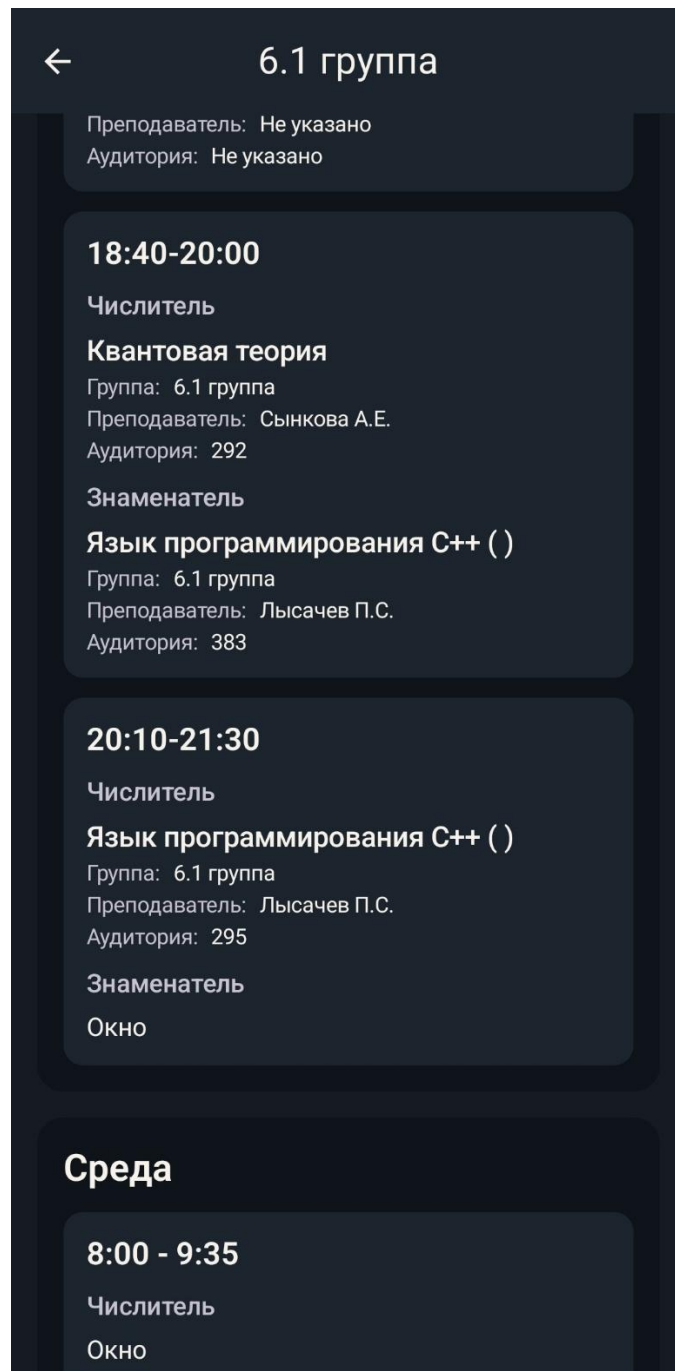


Рисунок 14 - Экран детального просмотра расписания (ScheduleDetailsScreen)

3.2.5 Боковое меню

Настройки и сведения об источнике вынесены в боковое меню HomeScreen. Переключатель в меню меняет тему и через ThemePreferencesRepository записывает выбор в SharedPreferences, так что настройка сохраняется между запусками.

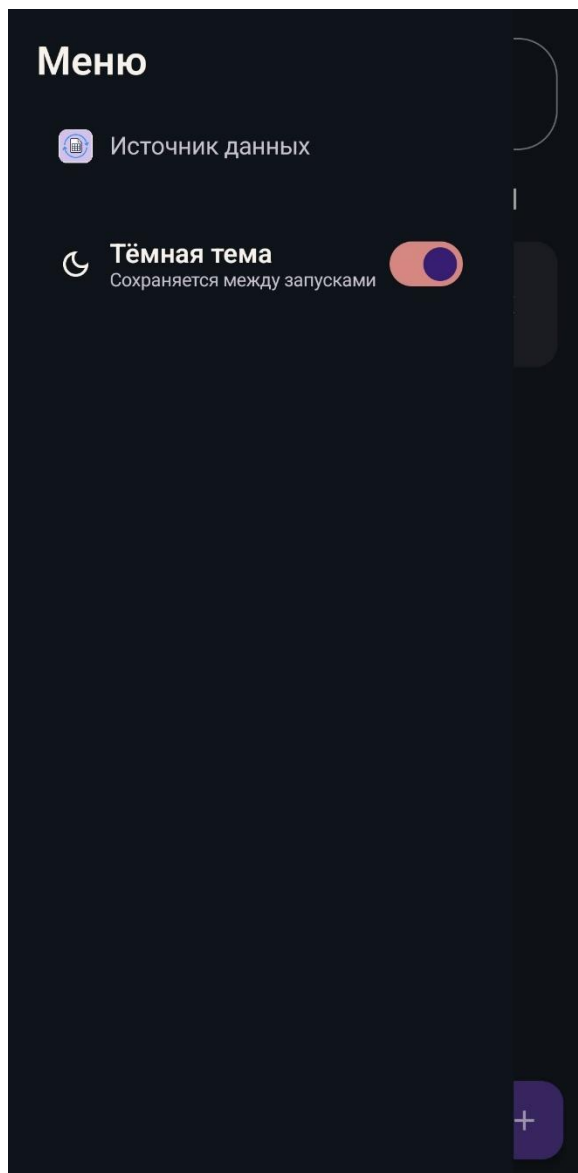


Рисунок 15 - Боковое меню

3.2.6 Раздел источник данных

В этом разделе показываются статус последнего импорта (MAPPED / REFRESHING / FAILED), ссылка на Google Sheets, метки времени проверки и загрузки, а также кнопка «Обновить данные».

Секция «Источник данных» показана на рисунке 14.

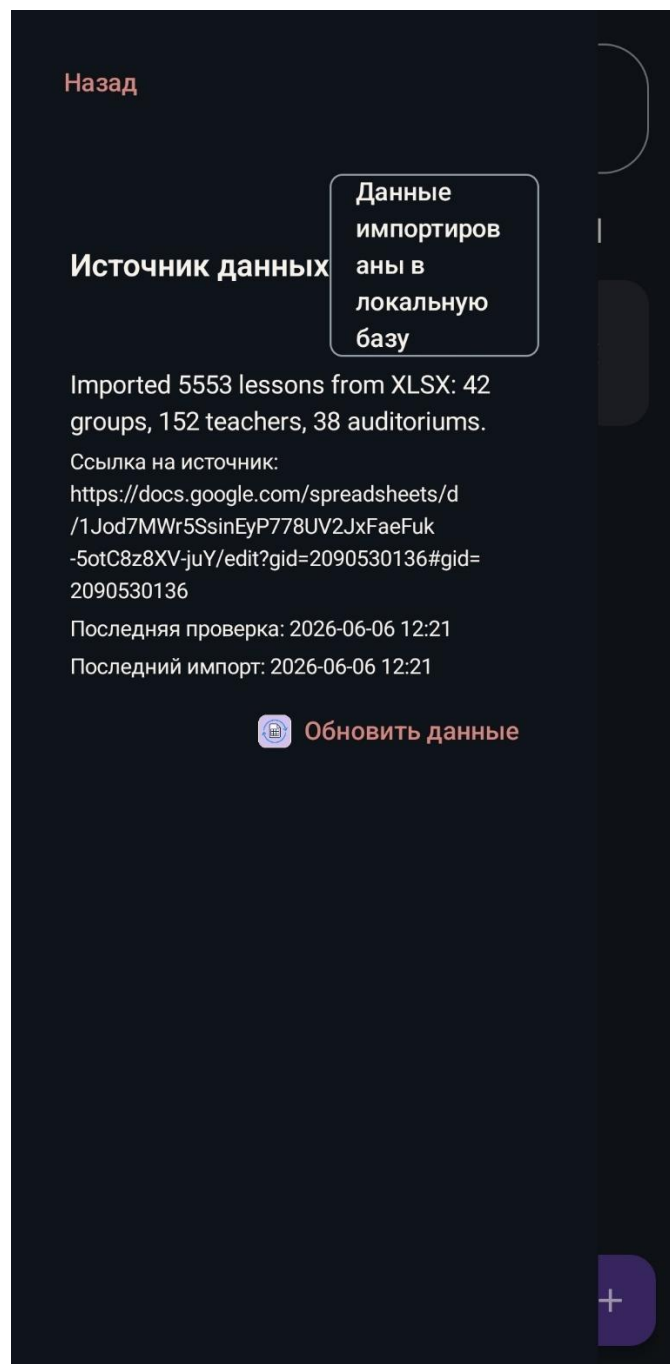


Рисунок 16 - Экран источника данных

Заключение

В результате выполнения курсовой работы создано мобильное Android-приложение «CS Schedule», которое даёт студентам и преподавателям факультета компьютерных наук ВГУ быстрый доступ к расписанию: данные подтягиваются из официальной таблицы автоматически, хранятся локально и открываются через персональные закладки.

Поставленные во введении задачи выполнены. Изучены предметная область и существующие аналоги; выяснилось, что ни одно из них не объединяет в одном клиенте три типа расписания, офлайн-режим и закладки. Спроектирована одноклиентная архитектура с модулем импорта, интерфейсом на Jetpack Compose и базой Room. Обоснован и применён стек Kotlin, Compose, Room, OkHttp, Jsoup, MVVM.

В коде реализован импорт с собственным парсером XLSX, учитывающим объединённые ячейки, чередование недель и пометки дистанционных пар. Собраны экраны списка закладок, детального расписания и управления источником данных; парсер покрыт модульными тестами.

«CS Schedule» можно использовать как готовый инструмент на факультете. Среди возможных улучшений на будущее — виджет на рабочий стол, уведомления об изменениях в таблице, поддержка нескольких факультетов и переключение между семестрами.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Расписание занятий ФКН ВГУ. — Текст: электронный // cs.vsu.ru: [сайт]. — URL: <https://www.cs.vsu.ru/rasp/> (дата обращения: 05.06.2026).
2. Kotlin Programming Language: Documentation. — Текст: электронный // kotlinlang.org: [сайт]. — URL: <https://kotlinlang.org/docs/home.html> (дата обращения: 10.04.2026).
3. Jetpack Compose UI App Development Toolkit. — Текст: электронный // developer.android.com: [сайт]. — URL: <https://developer.android.com/compose> (дата обращения: 12.04.2026).
4. Guide to app architecture. — Текст: электронный // developer.android.com: [сайт]. — URL: <https://developer.android.com/topic/architecture> (дата обращения: 15.04.2026).
5. Room Persistence Library. — Текст: электронный // developer.android.com: [сайт]. — URL: <https://developer.android.com/training/data-storage/room> (дата обращения: 18.04.2026).
6. Navigation with Compose. — Текст: электронный // developer.android.com: [сайт]. — URL: <https://developer.android.com/develop/ui/compose/navigation> (дата обращения: 25.04.2026).
7. Coroutines: Kotlin Documentation. — Текст: электронный // kotlinlang.org: [сайт]. — URL: <https://kotlinlang.org/docs/coroutines-overview.html> (дата обращения: 28.04.2026).
8. OkHttp: An HTTP client for Android and the JVM. — Текст: электронный // square.github.io: [сайт]. — URL: <https://square.github.io/okhttp> (дата обращения: 20.04.2026).
9. Jsoup: Java HTML Parser. — Текст: электронный // jsoup.org: [сайт]. — URL: <https://jsoup.org> (дата обращения: 22.04.2026).

10. Office Open XML File Formats. — Текст: электронный // learn.microsoft.com: [сайт]. – URL: <https://learn.microsoft.com/en-us/office/open-xml/structure-of-a-spreadsheetml-document> (дата обращения: 01.05.2026).

11. Google Sheets API: Concepts. — Текст: электронный // developers.google.com: [сайт]. – URL: <https://developers.google.com/sheets/api/guides/concepts> (дата обращения: 05.05.2026).

12. Material Design 3. — Текст: электронный // m3.material.io: [сайт]. – URL: <https://m3.material.io> (дата обращения: 10.05.2026).

13. Robolectric: Android Unit Testing Framework. — Текст: электронный // robolectric.org: [сайт]. – URL: <https://robolectric.org> (дата обращения: 15.05.2026).